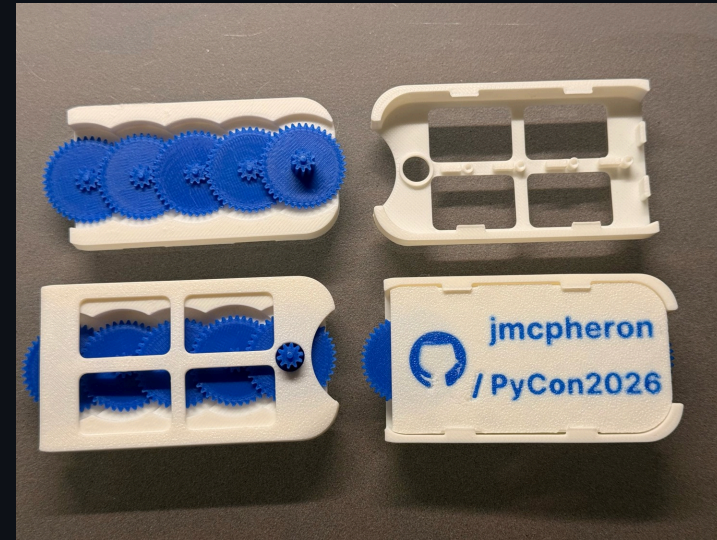


I Made a Gearbox Business Card, Then Made Python Explain It

Jason McPheron · PyCon US 2026 · Lightning Talk

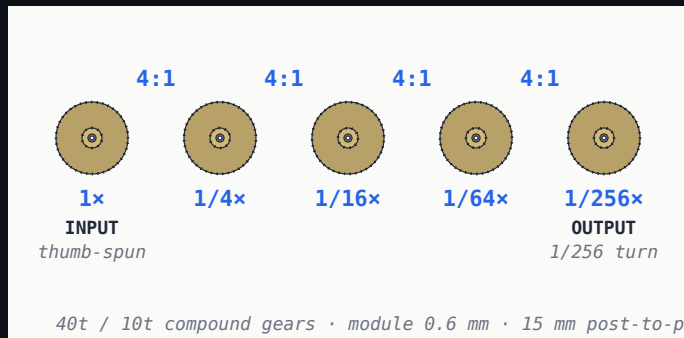
A 3D-Printed Business Card

- First PyCon — needed a conversation starter
- Credit-card size: 88.9 × 50.8 × 5mm
- Two printed halves, five compound gears
- Input: your thumb. Output: barely moves.



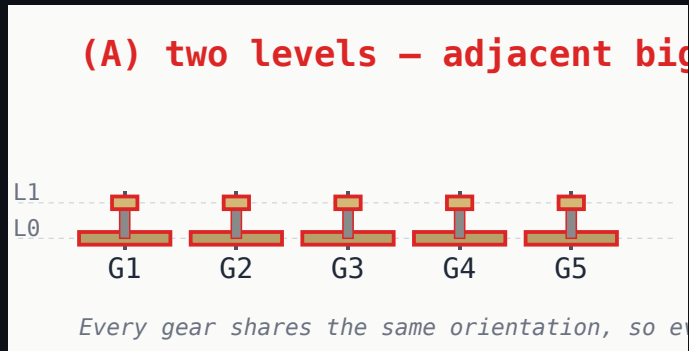
$4:1 \times 4:1 \times 4:1 \times 4:1 = 256:1$

- Big gear: **40 teeth**. Pinion: **10 teeth**. Each mesh = **4:1**
- Four meshes $\rightarrow 4^4 = \mathbf{256:1}$ total
- ~45 metres of thumb travel per output revolution



Puzzle 1 — How Do You Keep It 5mm Thick?

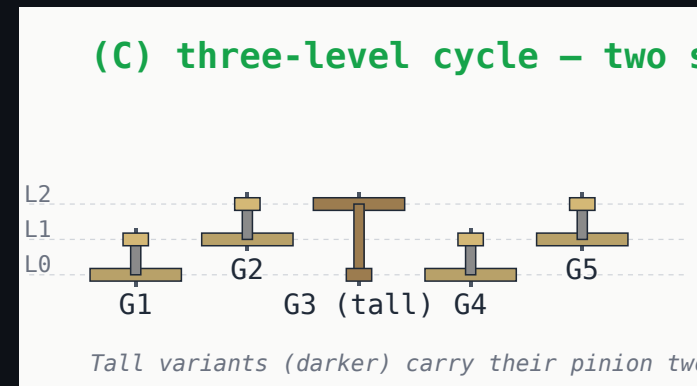
Naive 2-level layout



Big discs collide at the mesh point

5 gears, 50 gears — **still 3 levels, still 5mm**

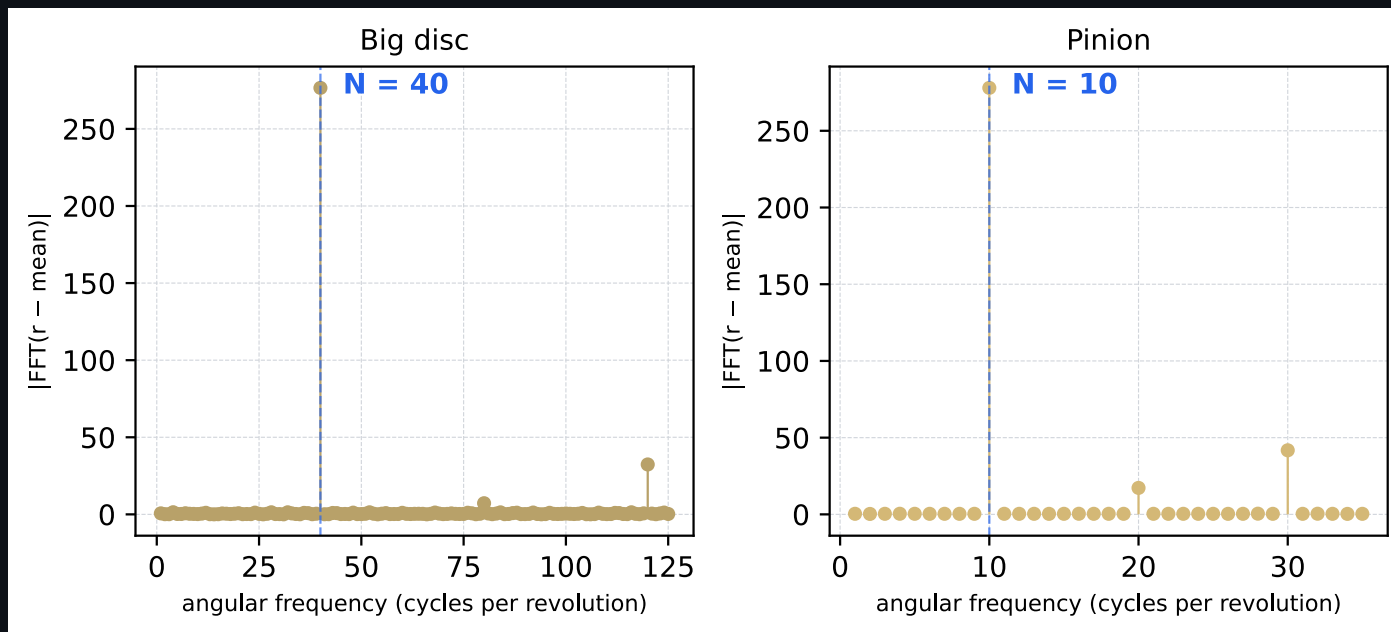
3-level cycle — the actual solution



Two standard gears + one tall-hub variant,
repeat

Puzzle 2 — Recover Tooth Count from Raw Geometry

- STEP export strips all parametric metadata — just B-rep solids
- Slice the STL at two heights → radial profile $r(\theta)$
- FFT the signal → dominant frequency = **tooth count**



Two Python Packages, One STEP File

cardlab — reads the STEP file

- **inspect** — AP242 assembly tree
- **render** — orthographic PNGs
- **explode** — per-part STL/GLB/GIF
- **spin** — gear chain animation

explainers — reads **card.py**

- SVG diagrams (drawsvg)
- Matplotlib charts
- Markdown pages
- CI auto-commits everything

One File Drives Everything

```
# src/explainers/card.py – the single source of truth
BIG_TEETH      = 40
PINION_TEETH   = 10
N_STAGES       = 5

RATIO_PER_STAGE = BIG_TEETH / PINION_TEETH           # 4.0
TOTAL_RATIO     = RATIO_PER_STAGE ** (N_STAGES - 1)  # 256.0
```

- Every SVG, every markdown page, every GIF reads from here
- Tests enforce: no hardcoded literals in explainer output
- Change `N_STAGES = 6` → ratio becomes **1024:1**, card still **5mm**

The Repo Explains Itself

`jmcpheron-card.step` → `cardlab` → `docs/assets/card/` (*renders, GIFs*)

`src/explainers/card.py` → `explainers build` → `docs/explainers/` (*SVGs, pages*)

`docs/` → `pages.yml` → **GitHub Pages**

- 3 GitHub Actions workflows — disjoint output paths, no conflicts
- Push a new STEP export → GIFs update within a minute
- Edit `card.py` → all explainer pages regenerate

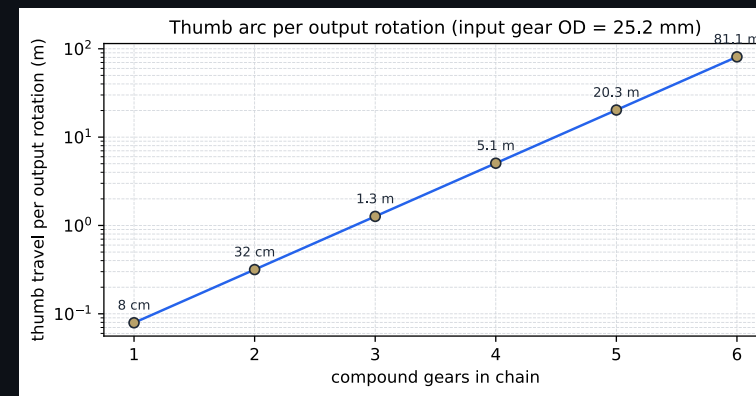
What Surprised Me

- Making the object was **easy**. Making Python **explain** the object was the fun part.
- FFT for tooth counting — no special libraries, just NumPy
- A 78-line constants file turned out to be a better design document than any CAD annotation I could have written

The design started in Onshape. Python came in afterward — around the edges. But that's where all the interesting stuff ended up.

Find Me on the Floor

- Python ↔ CAD pipelines (STEP → build123d → OpenSCAD → GIF)
- Print-in-place geometry tricks
- Open-source licensing for physical 3D files
(CC BY-SA — *did I get that right?*)



github.com/jmcpheeron/pycon2026



I have cards — ask for one.